代码承载生命与职业底线

软件工程的伦理与敬畏



道：伦理与敬畏 | 术：标准与工具

代码不是冰冷的符号

软件工程的每一行代码都绝非冰冷的逻辑符号，而是可能承载着千钧生命的重量。在当今数字化世界中，代码已经渗透到人类生活的每一个角落。从医疗设备的生命监护，到交通系统的精准调度，再到金融交易的毫秒决策，代码已成为现代文明的神经系统。





2018.10.29

2019.3.10

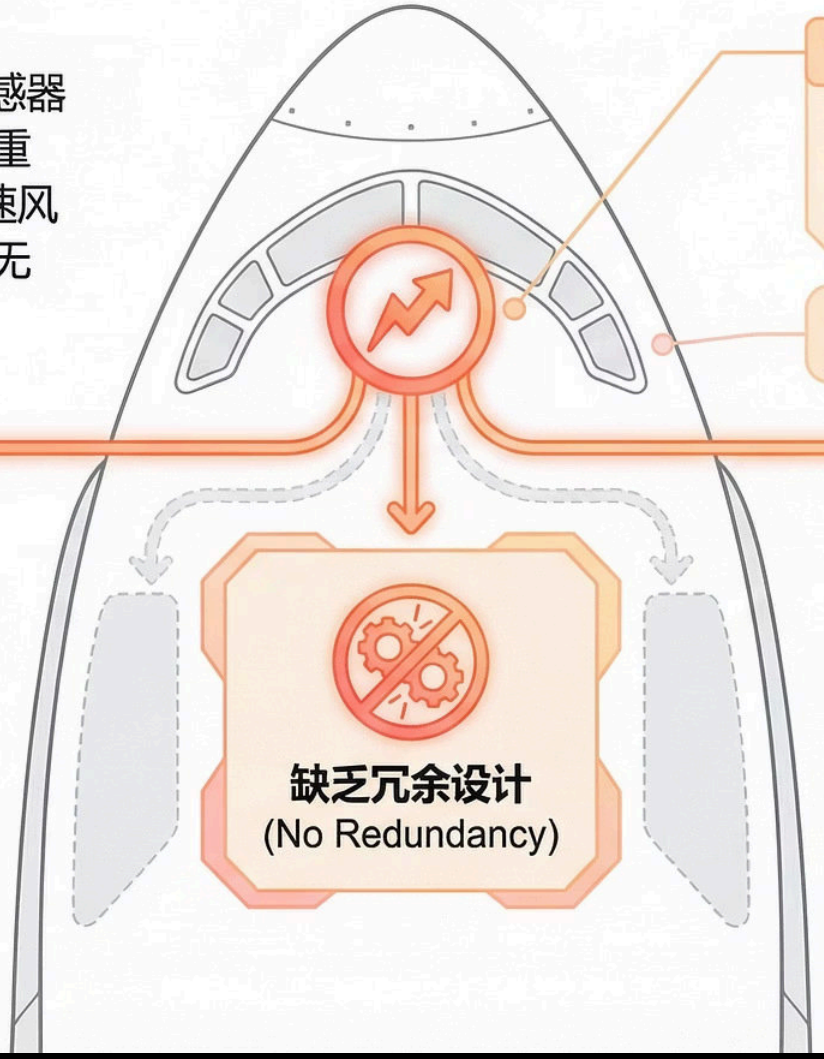


波音737 MAX：346条生命的代价

2018年10月29日印尼狮航和2019年3月10日埃塞俄比亚航空，两架波音737 MAX客机相继坠毁，346条鲜活生命骤然消逝。调查揭示，MCAS（机动特性增强系统）的软件逻辑缺陷成为致命杀手。这不是天灾，而是可以避免的人祸。

致命的系统缺陷

MCAS系统的致命漏洞在于：单一传感器依赖，缺乏冗余设计，飞行员培训严重不足。　　当迎角传感器错误报告失速风险时，系统自动压低机头，且飞行员无法快速识别并覆盖系统控制。



CRITICAL SPECIFICATIONS:

Sensor Type: Single AOA (Angle of Attack);
Activation Trigger: High AOA data;
System Authority: Automatic nose-down trim;

Pilot Override: Complex, non-intuitive procedures.

单一传感器依赖
(Vulnerability)

缺乏冗余设计
(No Redundancy)

飞行员培训严重不足
(Inadequate Training)

丰田刹车门：失控的算法

2009-2010年间，丰田因电子油门控制系统的软件逻辑错误被迫召回超过800万辆汽车。意外加速现象导致多起致命交通事故，调查发现软件在处理刹车与油门信号时存在竞态条件和内存溢出问题，使驾驶员完全失去对车辆的控制。

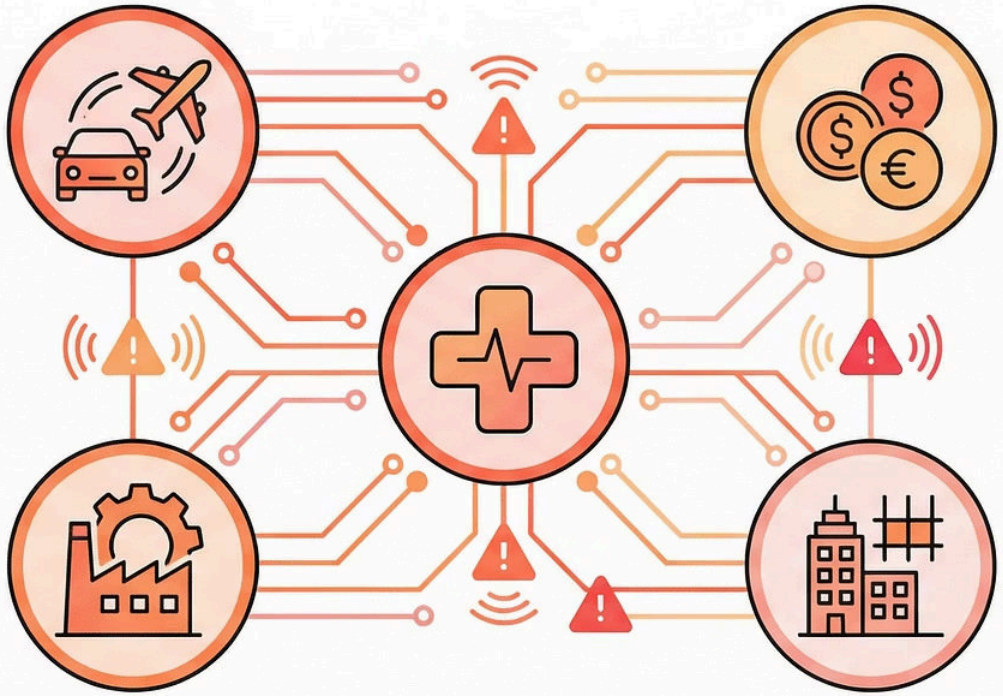


纪念受害者：愿逝者安息，悲剧不再重演。

软件缺陷的社会成本

停机时间：>5000小时

召回数量：>100万件



受影响人数：>50万人

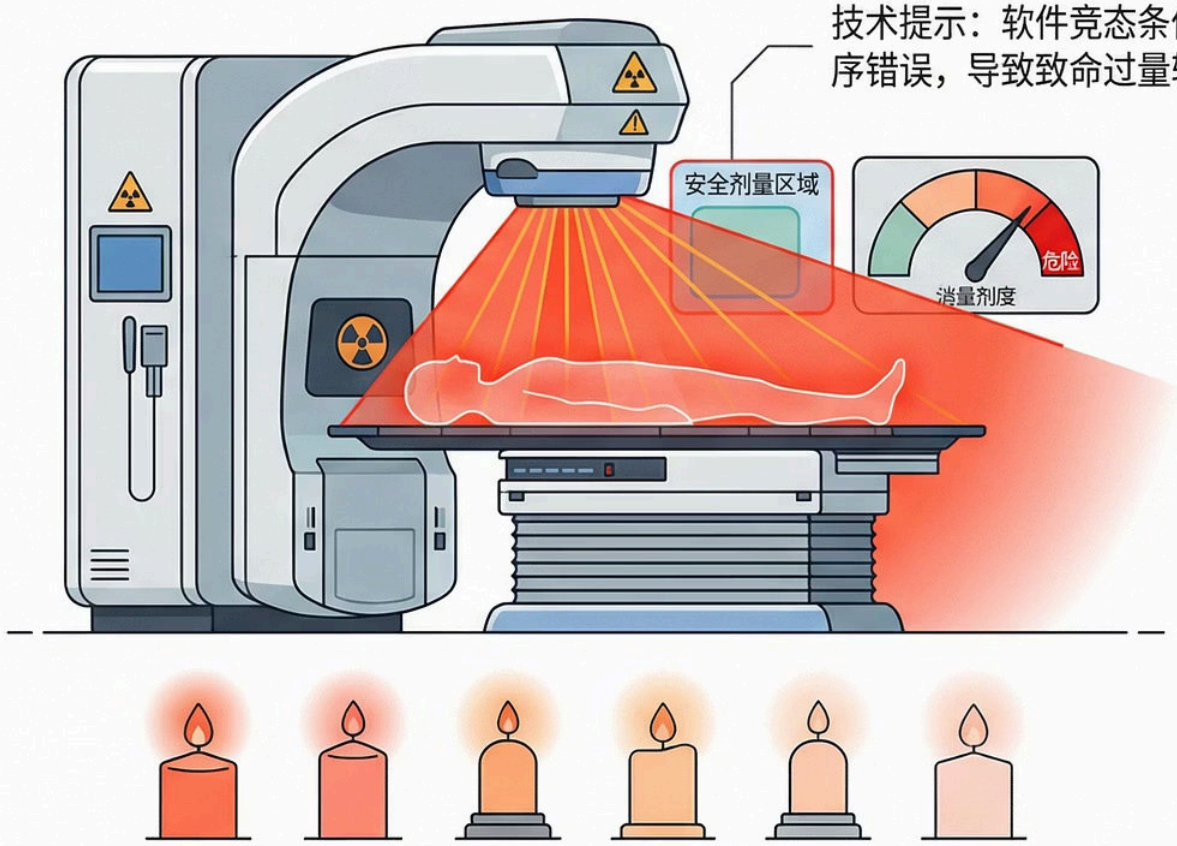
受影响人数：>50万人

每一个软件错误都可能引发连锁反应。从工厂自动化系统故障导致的生产停滞和工人伤亡亡，到智能电网故障造成的大规模停电，再到自动驾驶系统失误引发的交通事故，软件缺陷的影响已超越单一产品，演变为系统性社会风险。

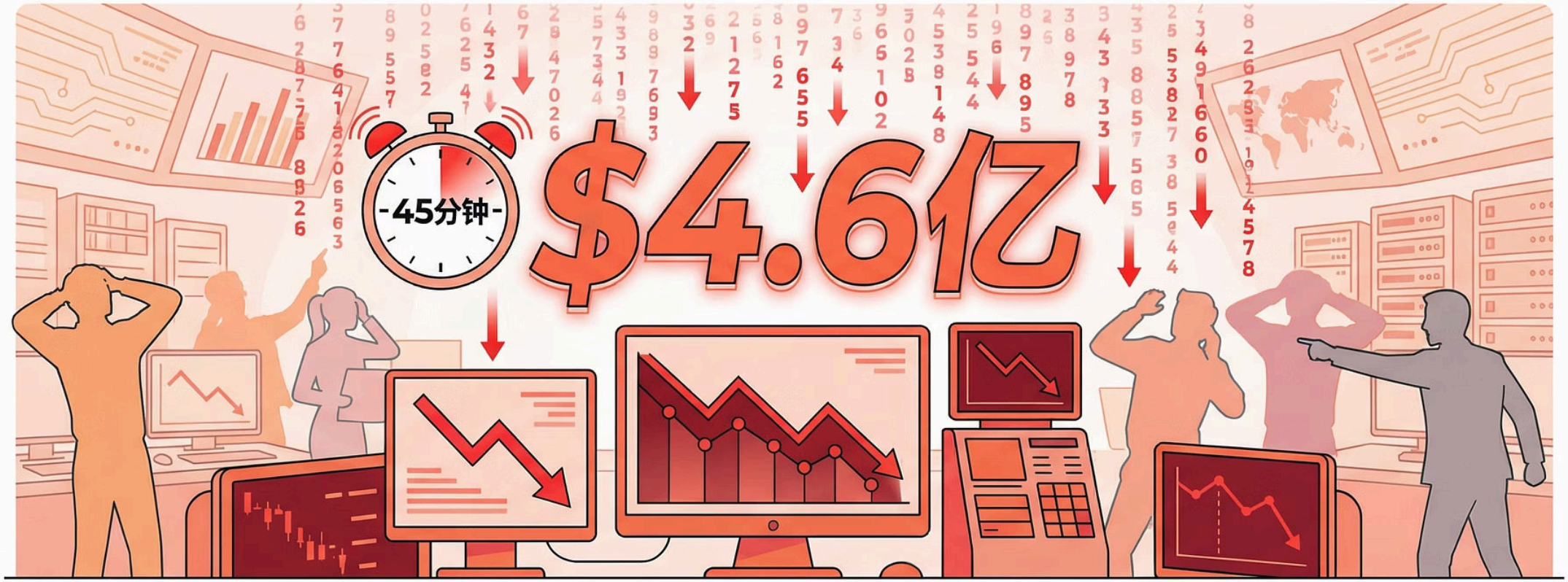
Therac-25：医疗软件的生死边界

Therac-25放射治疗设备因软件竞态条件缺陷，在1985-1987年间导致至少6名癌症患者接受了致命剂量的辐射。软件在处理治疗模式切换时存在时序漏洞，使患者在毫无察觉中被过量照射，这些bug直接夺取了本应被治愈的生命。

技术提示：软件竞态条件缺陷及模式切换时序错误，导致致命过量辐射。



金融系统的蝴蝶效应



2012年8月1日，骑士资本公司因算法交易系统部署错误，仅用45分钟就损失4.6亿美元，几乎导致公司破产。一个未被测试的代码分支意外激活，疯狂发送错误交易指令。这个案例警示我们：一行错误代码可能引发全球金融市场的剧烈震荡。

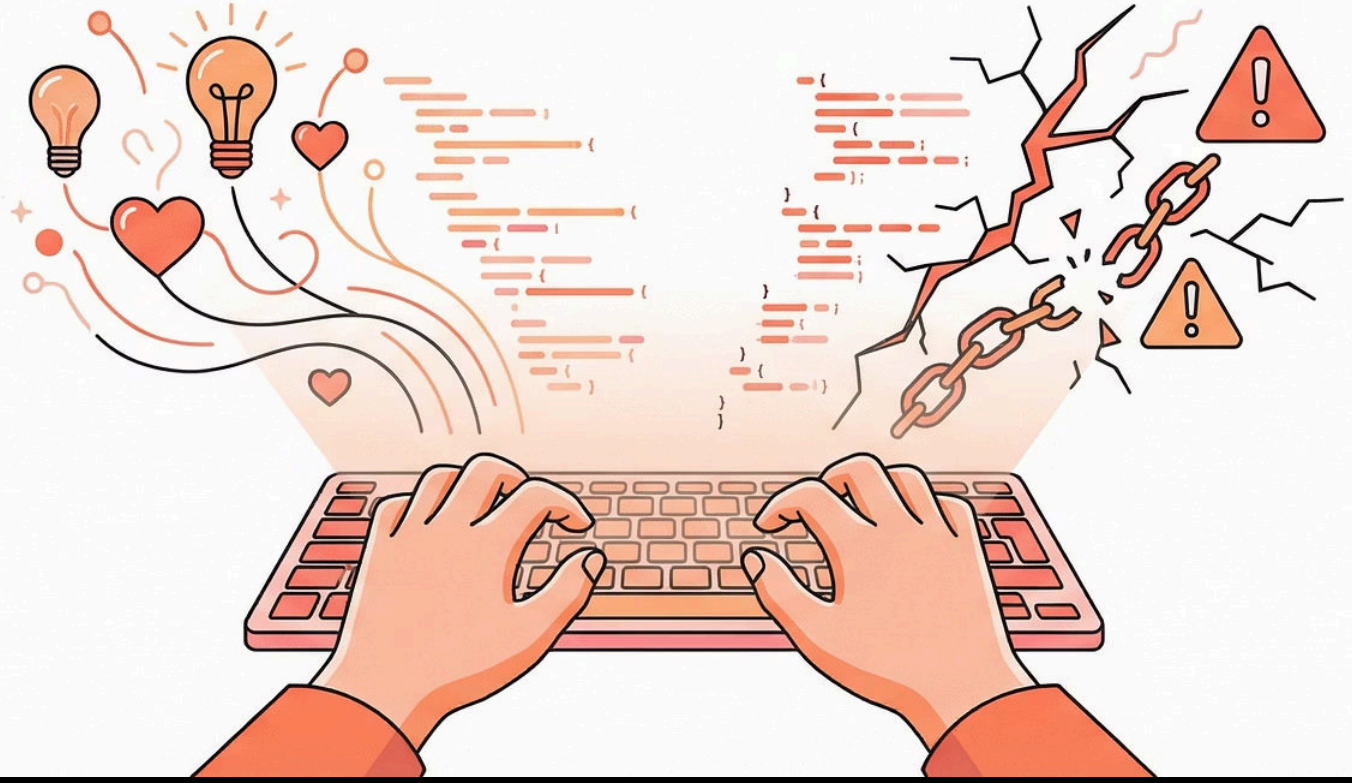
原型系统：职业伦理的演练场

在我们的教学中，原型开发阶段被视为职业伦理的重要演练场。我们明确告诉学生：即使是原型设计，也绝能成为随意堆砌垃圾代码的借口。原型阶段形成的编码习惯会伴随工程师一生，糟糕的开始只会导致更糟的结果。每个原型都应体现专业标准和责任意识。



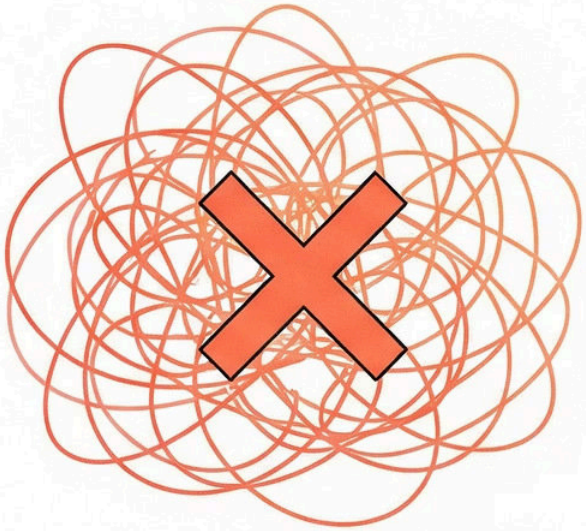
建立对代码的敬畏之心

每一位未来的软件工程师都必须建立对代码的敬畏之心。深刻理解手中的键盘不仅能创造改变世界的价值，若缺乏责任感与严谨的测试验证，同样能制造毁灭性的灾难。敬畏不是恐惧，而是对代对代码影响力的深刻认知和对自身责任的清醒把握。

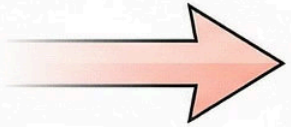


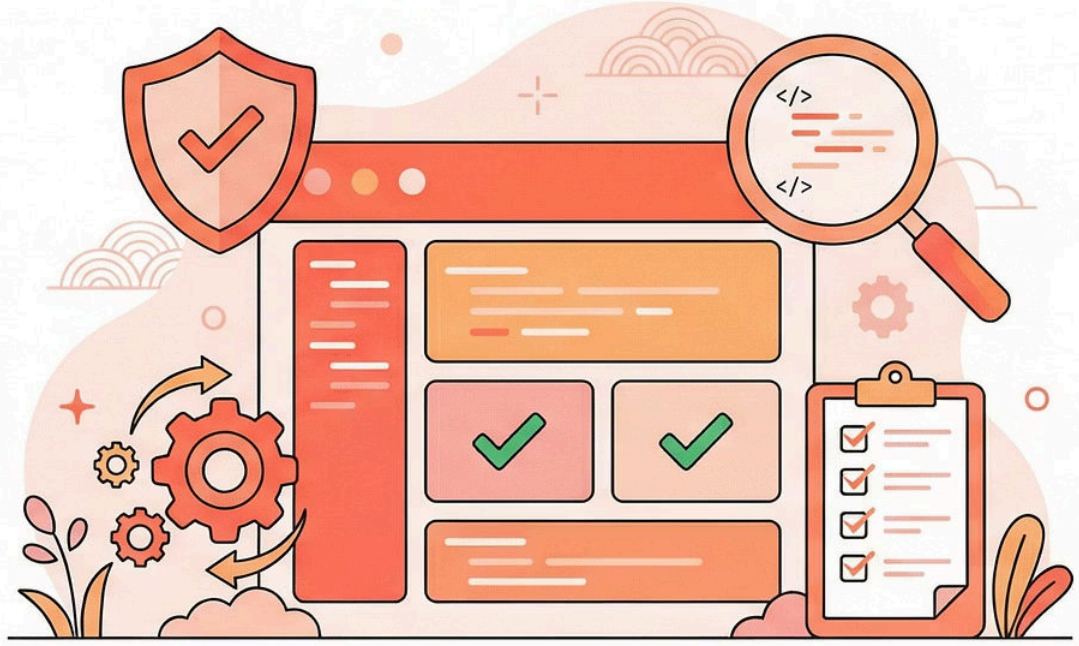
工程规范与科学闭环

保障代码质量不能仅靠开发者的个人良心，更需依赖严谨的工程科学体系。我们坚决摒弃.



作坊式

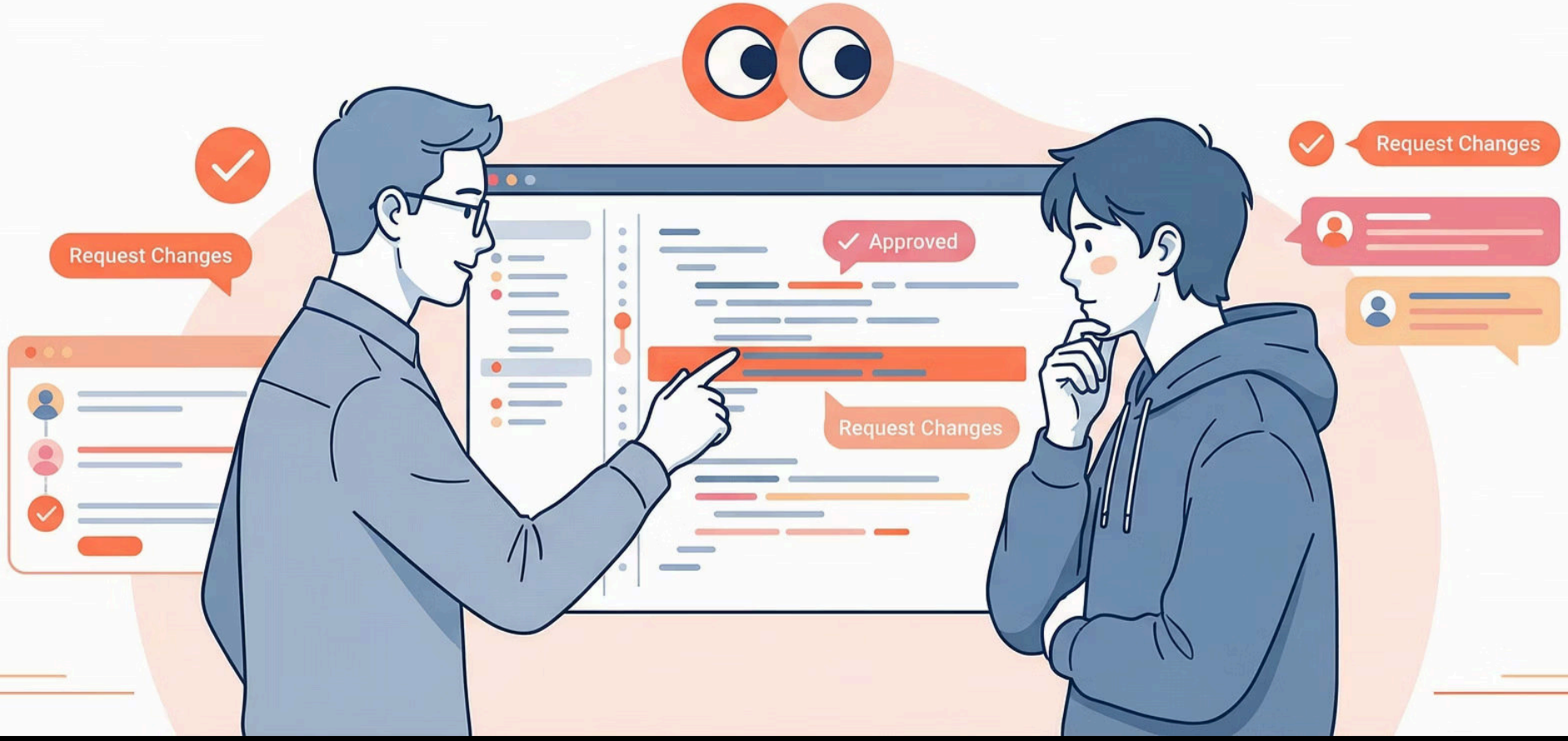




工程化

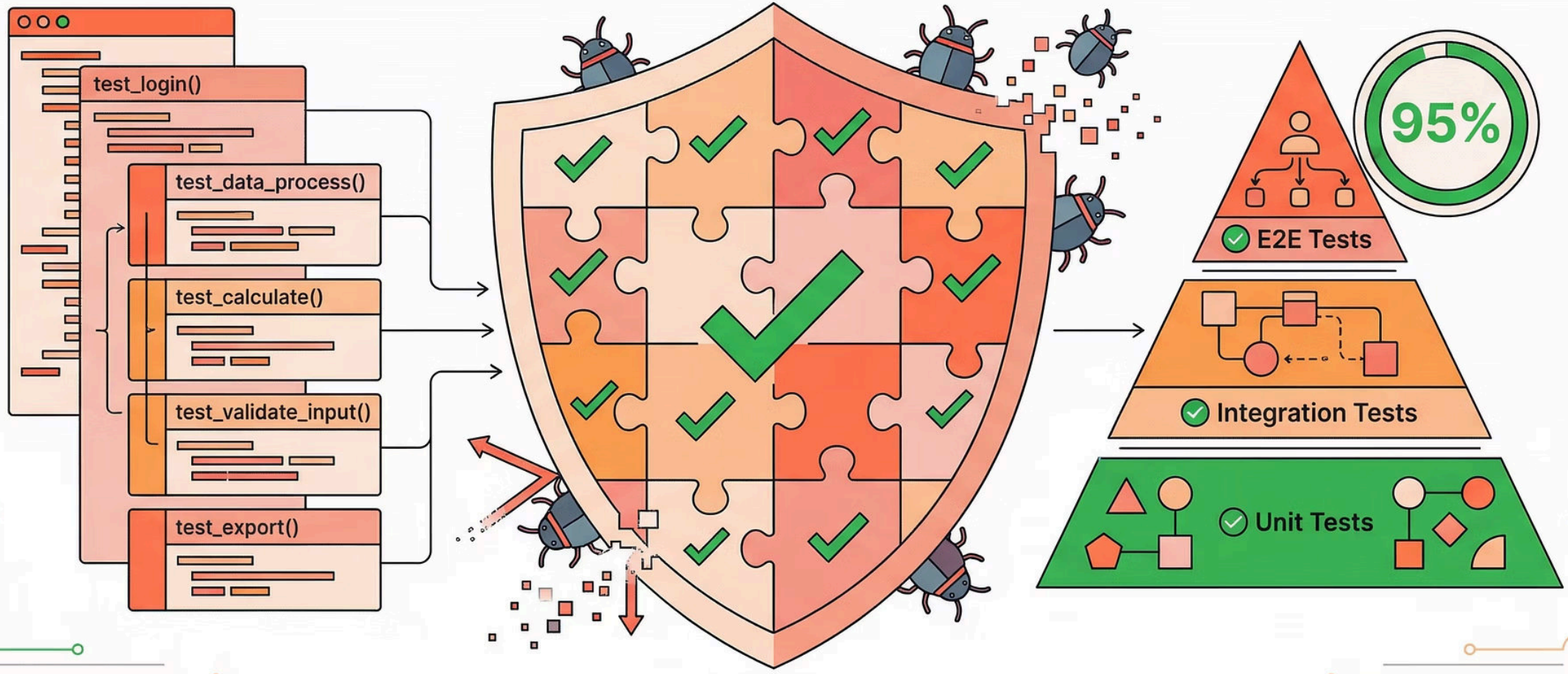
代码审查：集体智慧的力量

我们强制引入代码审查制度，践行四眼原则确保代码质量。每一行代码在合并到主分支前都必须经过至少一位同行的严格审视。Code Review不仅能发现潜在bug和逻辑缺陷，更能传播最佳实践、统一编码风格，并培养团队成员的质量意识和责任共担精神。



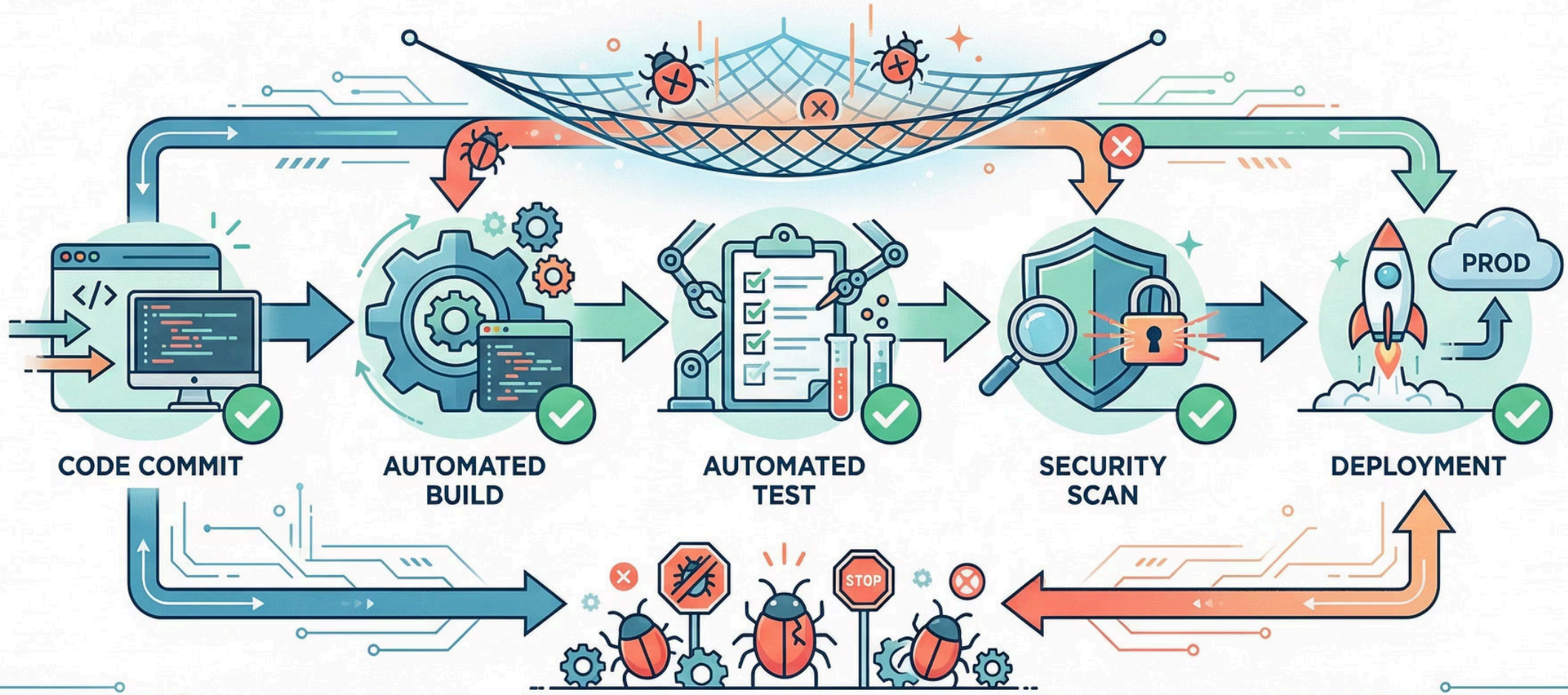
自动化单元测试的护盾

自动化单元测试是代码质量的第一道防线。测试驱动开发（TDD）理念要求先编写测试用例再实现功能，让潜在的bug在萌芽阶段就被扼杀。每个函数、每个模块都应该有对应的测试覆盖，形成完整的质量防护网，使代码在任何修改后都能快速验证其正确性。



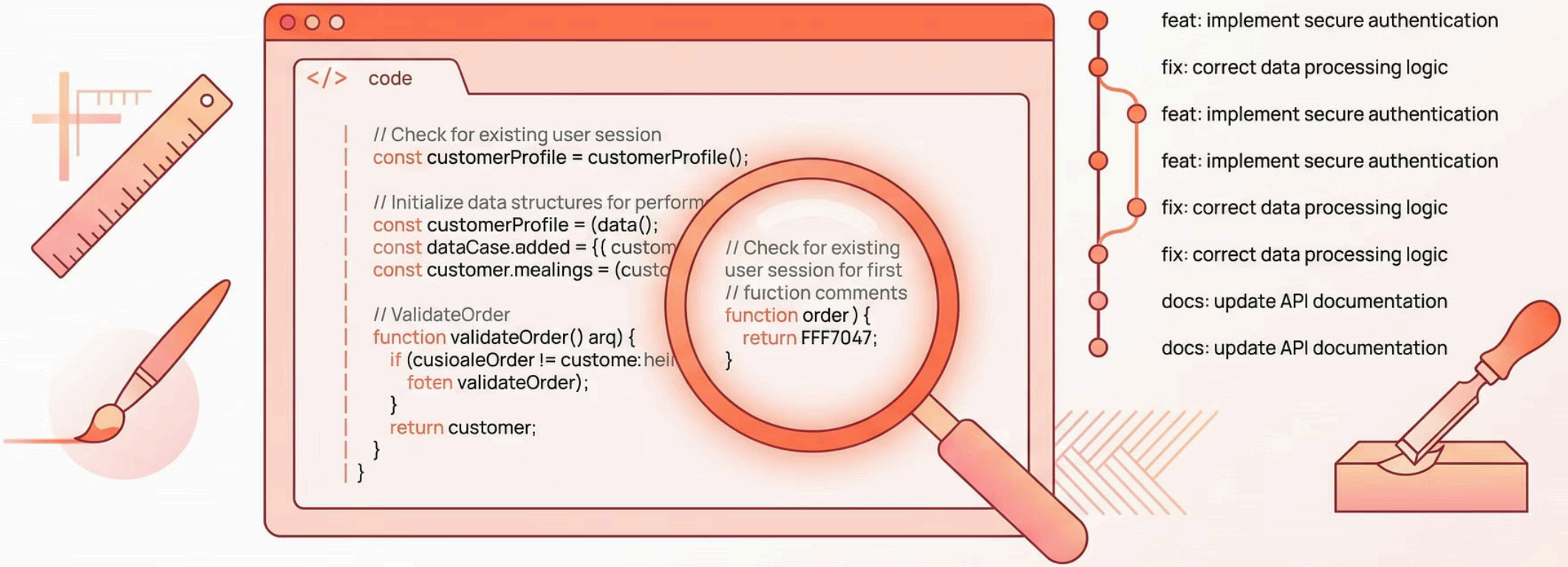
CI/CD：持续集成的安全网

持续集成/持续部署（CI/CD）确保每次代码变更都经过完整的自动化验证流程。从代码提交触发自动构建、运行完整测试套件、执行代码质量检查到自动部署，整个流水线让人为疏漏无处遁形。任何破坏性变更都会在进入生产环境前被拦截。



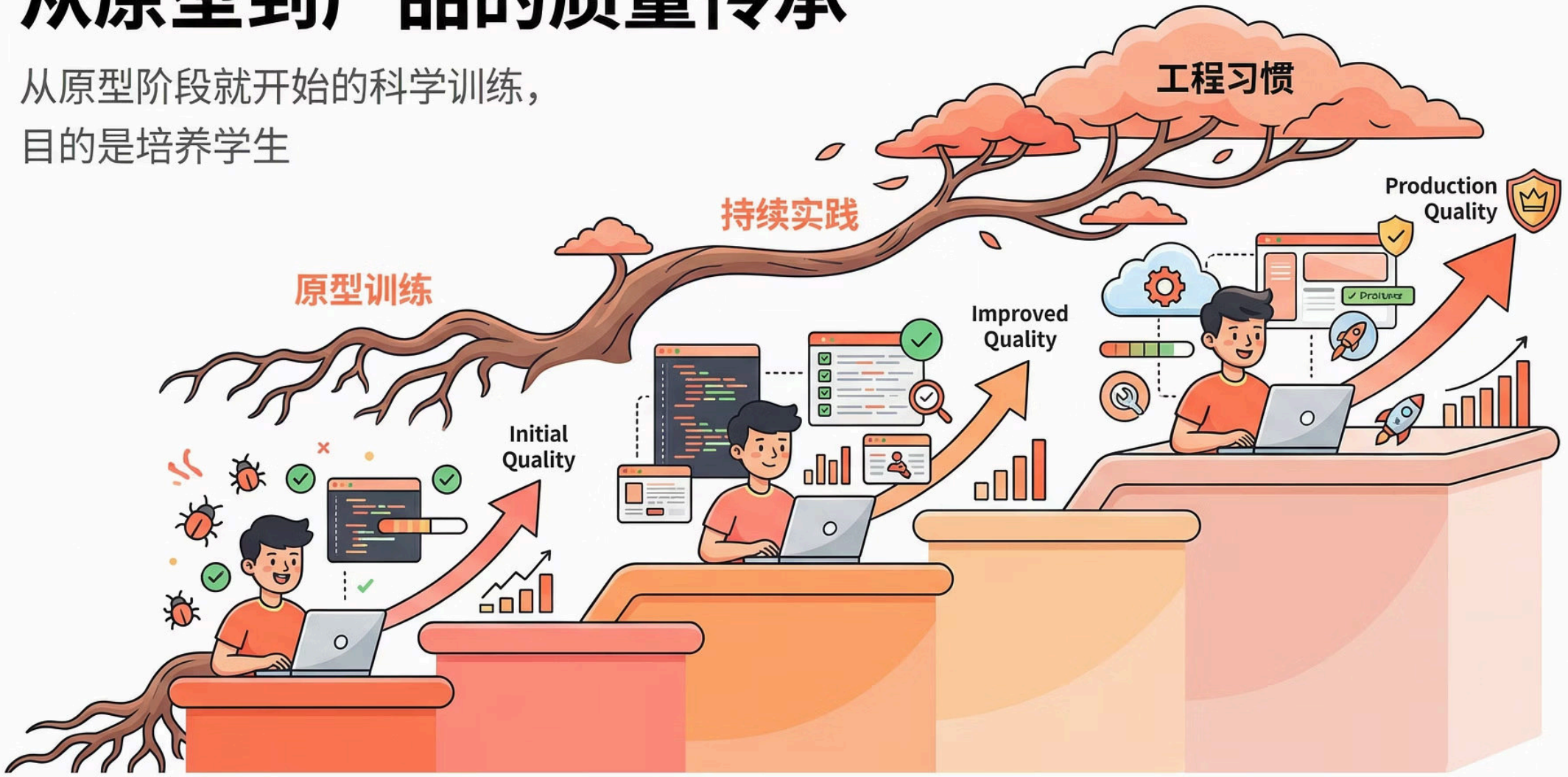
规范化开发的工匠精神

规范的命名约定、详尽的注释文档、严格的Git版本控制，这些看似繁琐的要求，实则是将抽象的工匠精神具象化为每一次代码提交的严谨与克制。变量命名要自解释，注释要说明意图而非重复代码，每次提交要原子化且附有清晰说明。细节决定品质。

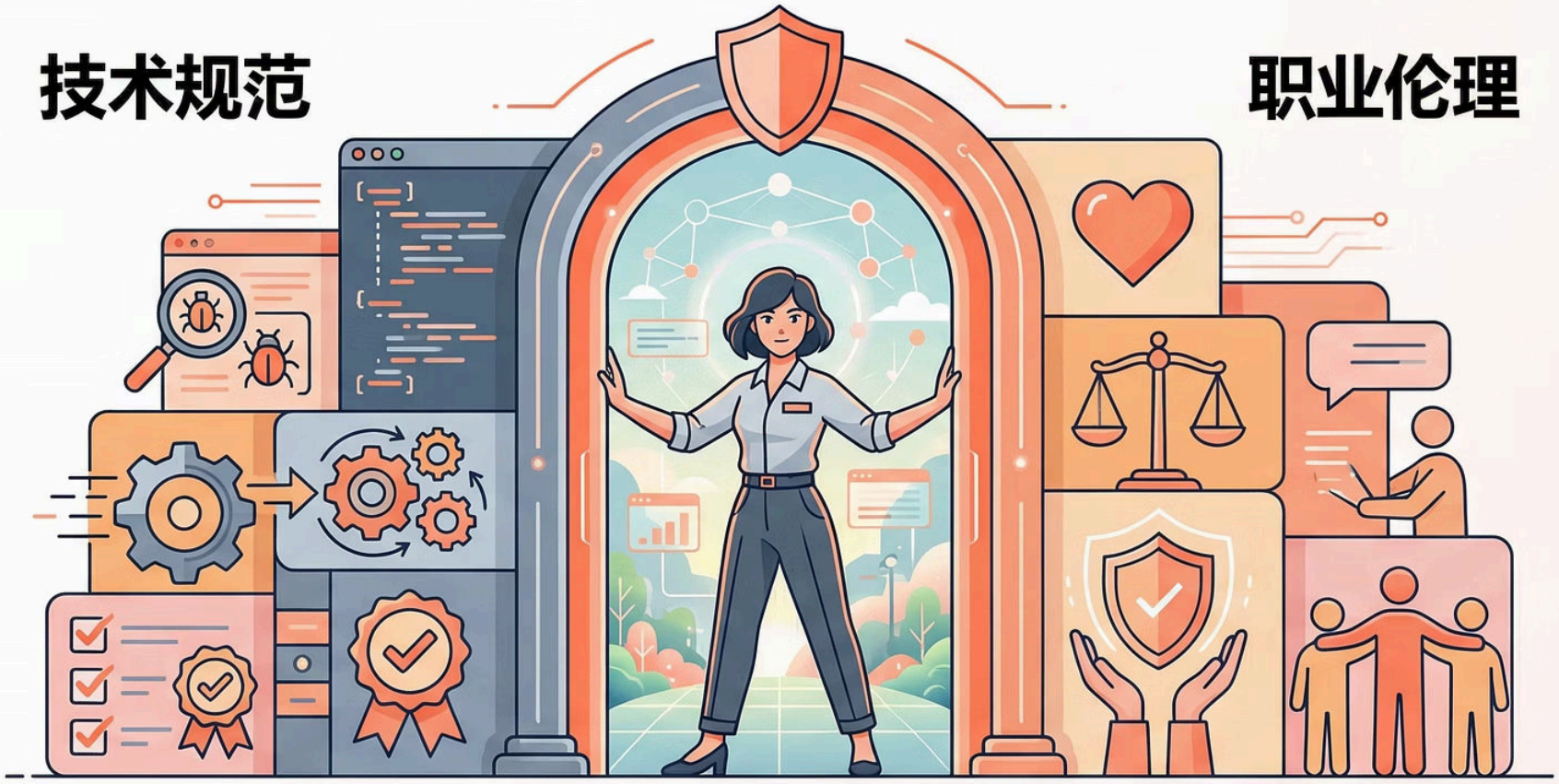


从原型到产品的质量传承

从原型阶段就开始的科学训练，
目的是培养学生



技术与伦理的完美融合



当严谨的技术规范遇上坚定的职业伦理，当自动化工具链遇上人性的责任担当，我们培养出的不仅是熟练的程序员，更是数字世界的守护者。技术提供方法，伦理指引方向，两者缺一不可。

每一位合格的软件工程师都应该成为守护数字世界安全的忠诚卫士。

代码承载未来，责任铸就品格

在这个由代码驱动的时代，软件已成为现代文明的基石。我们不仅要做技术的创造者，更要做责任的承担者。每一行代码都是对未来的承诺，每一次测试都是对生命的尊重。让我们以敬畏之心书写代码，用工程之术保障质量，共同守护这个数字化世界的安全与美好。

